

Activity 2 - Validate a product code with strings

AI Vibe Coding with Rust / TGS-2023039924 / version 3.1 / K1/A1

Specification

`is_valid_sku(&str) -> bool`: after trimming, exactly 3 ASCII upper-case letters, "-", 4 ASCII digits.
`normalise_sku(&str) -> Option<String>`: trimmed upper-case SKU when valid, else None.

Part 1: Learn Rust prompts

Explain it simply

I am learning Rust. Explain String versus &str, trimming, UTF-8 bytes versus chars, and validating text to a beginner in plain English. Use a short example from a small shop's stock system, keep the code under 20 lines, show its output, and point out one mistake beginners often make.

Walk me through the sample

Here is `samples/01_string_and_str.rs` from my course. Walk through it line by line. Before I run it, ask me to predict the output. Then suggest one small change I can make to experiment, and tell me what should happen.

Quiz me

Quiz me on String versus &str, trimming, UTF-8 bytes versus chars, and validating text with 5 questions, one at a time. Wait for my answer each time and explain why it is right or wrong. Include one question where I predict the output or the compiler error of a short Rust snippet.

Compare with what I know

I know some Python or JavaScript. In a small table, compare how Rust handles strings and text with Python and JavaScript. Then show one Rust example and explain what the Rust compiler checks for me that the others do not.

Part 2: Build with AI (vibe-coding prompts)

Plan before code

Act as a senior Rust reviewer. Specification: `is_valid_sku(&str) -> bool`: after trimming, exactly 3 ASCII upper-case letters, "-", 4 ASCII digits. `normalise_sku(&str) -> Option<String>`: trimmed upper-case SKU when valid, else None. Before writing any code, list the public signatures, the edge cases and at least four boundary tests you would expect. Ask me about anything ambiguous.

Generate with AI

Implement "Validate a product code with strings" in `starter/src` so that the supplied tests pass. Borrow &str inputs, return an owned String only from `normalise_sku`, and do not use regex crates. Keep public signatures unchanged, use no unsafe code, and explain each design decision in one sentence.

Repair from evidence

Here is my exact cargo output: `<paste the first error or failing test>`. Explain the cause in plain English, propose the smallest fix that keeps the specification, and add one regression test. Do not change

expected values in existing tests.

Review checklist

Signatures unchanged; no unsafe; no unwrap() on input; no new crates; every boundary tested; cargo test, clippy -D warnings and fmt --check pass; you can explain every line.

Part 3: Rust sample scripts (samples/ folder)

01_string_and_str.rs

Run: `rustc --edition 2021 01_string_and_str.rs && ./01_string_and_str`

```
// Sample: owned String versus borrowed &str, and bytes versus characters.
fn describe(text: &str) {
    println!(
        "{text:?}: {} bytes, {} chars",
        text.len(),
        text.chars().count()
    );
}

fn main() {
    let owned: String = String::from(" abc-1234 ");
    let trimmed: &str = owned.trim(); // borrows part of `owned`
    let upper: String = trimmed.to_ascii_uppercase(); // new owned text
    describe(&owned);
    describe(trimmed);
    describe(&upper);
    describe("ÄBC-1234");
}
```

Expected output:

```
" abc-1234 ": 12 bytes, 12 chars
"abc-1234": 8 bytes, 8 chars
"ABC-1234": 8 bytes, 8 chars
"ÄBC-1234": 9 bytes, 8 chars
```

02_chars_and_validation.rs

Run: `rustc --edition 2021 02_chars_and_validation.rs && ./02_chars_and_validation`

```
// Sample: validating a product code character by character.
fn is_valid_sku(code: &str) -> bool {
    let chars: Vec<char> = code.trim().chars().collect();
    chars.len() == 8
    && chars[..3].iter().all(|c| c.is_ascii_uppercase())
    && chars[3] == '-'
    && chars[4..].iter().all(|c| c.is_ascii_digit())
}

fn main() {
    for code in [
        "ABC-1234",
        " ABC-1234\n",
        "abc-1234",
        "ABC-12A4",
        "ÄBC-1234",
    ] {
        println!("{code:?} valid = {}", is_valid_sku(code));
    }
}
```

Expected output:

```
"ABC-1234" valid = true
" ABC-1234\n" valid = true
"abc-1234" valid = false
"ABC-12A4" valid = false
"ÄBC-1234" valid = false
```

Learn more

Rust Book: Storing UTF-8 Encoded Text with Strings:

<https://doc.rust-lang.org/book/ch08-02-strings.html>

W3Schools: Rust Strings: https://www.w3schools.com/rust/rust_strings.php

Programiz: Rust String: <https://www.programiz.com/rust/string>

Rust official learning hub (the Book, Rustlings, Rust by Example): <https://rust-lang.org/learn/>

W3Schools Rust tutorial: <https://www.w3schools.com/rust/>

W3Schools: Get Started with Rust: https://www.w3schools.com/rust/rust_getstarted.php